

ClapCap: CLAP Prefix for Music Captioning

Team Members:
Ali Ozkaya 280201092

Advisor: Prof. Dr. Aytuğ Onan

A Thesis Report Submitted to the
Faculty of Engineering in Partial Fulfilment of the
Requirements for the Degree of

BACHELOR OF SCIENCE

(Academic Project)

Department of Computer Engineering
İzmir Institute of Technology
İzmir, Turkey
June, 2026

Abstract

This project studies whether a lightweight language model can describe music when fed nothing but a short, learned prefix projected from a frozen audio embedding. The pipeline, CLAPCAP, is the audio analogue of the ClipCap image-captioning recipe: a frozen CLAP audio encoder produces a single clip embedding, a small projection network maps it to a fixed-length prefix of pseudo-tokens, and a pretrained language model generates the caption. On the MusicCaps dataset we run a controlled comparison of two language-model families — GPT-2 (decoder-only, 124M parameters) and T5-base (encoder-decoder, 220M) — holding the encoder, projection, data split and evaluation fixed so that the language model is the only variable. We report a ten-metric evaluation suite with FENSE as the primary score, and we measure a per-model noise floor from three training seeds so that every ablation delta can be judged against run-to-run variance. We find that (i) the two models are statistically indistinguishable at baseline once the noise floor is accounted for; (ii) second-stage fine-tuning erases nearly all architectural differences introduced in the projection stage; (iii) the largest decoding lever, sentence completion, improves FENSE mostly by removing the fluency-error term rather than by improving semantics; and (iv) the models capture genre, instrumentation and mood well but share a systematic male-vocalist bias, which we trace to caption-prior amplification under uncertain audio evidence rather than to a missing audio cue. The work characterises the minimum viable architecture for music captioning; every experiment runs on a single consumer GPU.

Keywords: Music Captioning, Audio–Language Models, Prefix Conditioning, CLAP, Controlled Ablation, Evaluation Metrics

Contents

1	INTRODUCTION	3
2	RELATED WORK	4
2.1	Prefix captioning	4
2.2	General audio captioning	4
2.3	Music captioning	4
2.4	Audio language models	4
3	RESEARCH METHODOLOGY	5
3.1	Overview	5
3.2	Audio encoder	5
3.3	Projection network	5
3.4	Language models	5
3.5	Two-stage training	5
3.6	Efficiency	6
4	EXPERIMENTAL STUDY	7
4.1	Experimental setup	7
4.2	Baseline comparison	7
4.3	Stage 1 vs. Stage 2	8
4.4	Architecture ablations	9
4.5	Decoding ablations	10
4.6	Qualitative analysis	13
5	CONCLUSIONS AND FUTURE WORK	15
5.1	Limitations	15
5.2	Future work	15
	ALIGNMENT WITH UN SUSTAINABLE DEVELOPMENT GOALS	16
	REFERENCES	17
A	APPENDIX	19
A.1	Reproducibility	19
A.2	Noise-floor detail	19
A.3	Stage-wise validation losses	19
A.4	Baseline metric tables	21
A.5	Vocalist-bias statistics	21
A.6	Full architecture-ablation metrics	22
A.7	Full decoding-ablation metrics	23

1. INTRODUCTION

Most audio captioning research targets *general* sound — environmental events and acoustic scenes — on AudioCaps [2] and Clotho [3]. Music is different: a useful music caption describes *musical* attributes such as genre, instrumentation, mood, tempo and structure rather than discrete sound events. This project asks a deliberately minimal question: *can a lightweight language model understand music semantics through a simple projection from a frozen audio embedding?*

Our approach, CLAPCAP, adapts ClipCap [1], which captions images by projecting a frozen CLIP embedding into a prefix of pseudo-tokens for GPT-2. We replace the image encoder with a frozen CLAP audio encoder [4], caption music instead of images, and — crucially — turn the single-LM recipe into a *controlled comparison* of language models. Because the encoder, projection, data split and evaluation are all shared, the language model is the only thing that changes between conditions.

The contributions of this project are:

- An audio adaptation of the ClipCap prefix-captioning recipe to music, with a single shared evaluation harness.
- A *noise-floor* methodology: per-model run-to-run variance from three seeds, so single-run ablation deltas are interpretable rather than anecdotal.
- A controlled comparison of a decoder-only (GPT-2) and an encoder–decoder (T5) language model, with ablations over projection architecture and decoding strategy.
- A qualitative and statistical analysis of a recurring vocalist bias, which we trace to caption-prior amplification under uncertain audio evidence rather than to the audio frontend alone.

This project does not claim state-of-the-art performance. Published captioning systems are evaluated on different datasets with different reference counts, so a numerical comparison would be meaningless (Chapter 2), and we do not attempt one. The contribution is the controlled analysis itself, together with the efficiency of the recipe — only a small projection (plus the language model) is trained.

The remainder of this report is organised as follows. Chapter 2 reviews related work on prefix captioning, audio captioning and music captioning. Chapter 3 describes the CLAPCAP architecture and the two-stage training methodology. Chapter 4 presents the experimental setup, the evaluation metrics and the results of the baseline comparison, architecture ablations, decoding ablations and bias analysis. Chapter 5 summarises the findings and outlines future work. The appendix provides full metric tables and reproducibility instructions.

2. RELATED WORK

2.1 Prefix captioning

ClipCap [1] maps a frozen CLIP image embedding to a constant-length prefix ($K=10$ tokens) consumed by GPT-2, training only a small mapping network. It offers two design points: a lightweight transformer mapper with a frozen LM, and an MLP mapper with a fine-tuned LM. CLAPCAP is the audio analogue of the MLP-mapper, fine-tuned-LM variant. Notably, ClipCap reports that this variant *overfits* as trainable capacity grows — an observation we recover, amplified, when scaling the LM (Chapter 5). In the audio domain, Pengi [6] likewise conditions a small LM on an audio-derived prefix, framing a range of audio tasks as text generation.

2.2 General audio captioning

Audio captioning is typically benchmarked on AudioCaps [2] and Clotho [3], which describe general sounds and provide multiple references per clip. These differ from music captioning in domain, number of references, and the metrics commonly reported (CIDEr/SPIDEr rather than the semantic FENSE). Cross-dataset numerical comparison is therefore not meaningful, and we avoid it.

2.3 Music captioning

MusicCaps [5] is a hand-curated set of 5,521 ten-second clips from AudioSet [7], each annotated by musicians with a multi-sentence caption and a list of musical aspects. LP-MusicCaps [8] addresses caption scarcity by generating pseudo-captions with an LLM over tag datasets and trains a convolutional-encoder/BART-decoder captioner. Such domain-specific captioners are a natural *reference* for CLAPCAP, but a fair comparison must run them on our exact test split and avoid the leakage that arises when a captioner is itself trained on MusicCaps; we leave this to future work.

2.4 Audio language models

End-to-end audio LLMs such as Qwen-Audio [9] and SALMONN [10] couple a large LM to an audio encoder and are trained on large instruction corpora. They are far heavier than the present recipe; we regard them as an upper-bound reference direction rather than a controlled comparison, since they bypass the frozen-encoder/learned-prefix design we study.

3. RESEARCH METHODOLOGY

3.1 Overview

A clip is encoded once by a frozen CLAP model into a d_a -dimensional embedding. A projection network f_θ maps this vector to a prefix $p_1, \dots, p_k \in \mathbb{R}^{d_{lm}}$ of k pseudo-tokens in the LM embedding space. The LM then generates the caption autoregressively, conditioned on the prefix (Figure 3.1).

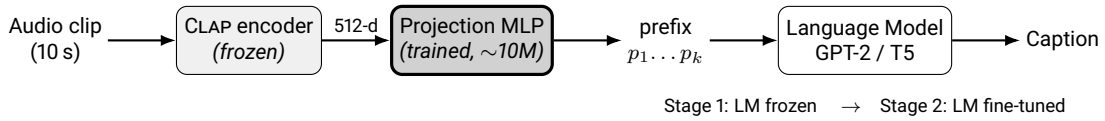


Figure 3.1: The CLAPCAP pipeline. A frozen CLAP encoder turns the audio clip into a single 512-d embedding; a small projection MLP (the only newly added module) maps it to a k -token prefix in the LM embedding space; the LM generates the caption. Stage 1 trains the projection with the LM frozen; Stage 2 additionally fine-tunes the LM.

3.2 Audio encoder

We use the LAION CLAP checkpoint `laion/clap-htsat-unfused` [4], whose audio tower is an HTS-AT transformer [11] producing a 512-dimensional audio embedding. CLAP is trained on general audio and is *not* music-specialised — a deliberate test of how far a general frontend carries on a music task, and a limitation we revisit. Embeddings are precomputed once and cached, so the encoder never runs during training or evaluation. The “unfused” variant ingests a fixed ~ 10 s window, matching MusicCaps clip length.

3.3 Projection network

The projection is a small MLP, $\text{Linear} \rightarrow \text{LayerNorm} \rightarrow \text{GELU} \rightarrow \text{Dropout} \rightarrow \text{Linear}$, reshaped to (k, d_{lm}) . The baseline uses prefix length $k=8$, dropout 0.3 and depth 2.

3.4 Language models

We compare GPT-2 [12] (decoder-only, 124M) and T5-base [13] (encoder-decoder, 220M). For T5 the prefix is fed to the encoder; for GPT-2 it is prepended as input embeddings. Extending the same harness to modern billion-parameter decoders is in progress (Chapter 5).

3.5 Two-stage training

In Stage 1 (S1) the LM is frozen and only the projection is trained; in Stage 2 (S2) the projection and the full LM are fine-tuned together (the audio encoder stays frozen). S1 corresponds to ClipCap’s frozen-LM regime and S2 to its fine-tuned-LM regime. The objective in both stages is autoregressive cross-entropy on caption tokens conditioned on the prefix,

$$\mathcal{L} = - \sum_j \log p_\theta(c_j \mid p_1, \dots, p_k, c_1, \dots, c_{j-1}).$$

We use AdamW [14] (weight decay 0.1), cosine annealing [15], batch size 8 and early stopping (patience 5). Per model, only the learning rates are tuned, via a 16-trial Optuna [16] search over an 8-epoch proxy; all else is fixed so the LM stays the controlled variable.

3.6 Efficiency

The only newly added module is the projection (~ 10.2 M parameters): Stage 1 trains it alone with the LM frozen, and Stage 2 additionally fine-tunes the LM. The complete model is 124M (GPT-2) or 220M (T5) parameters — one to two orders of magnitude smaller than the billion-parameter end-to-end audio LLMs — and every experiment in this report runs on a single consumer GPU (NVIDIA RTX 5090). Because CLAP embeddings are precomputed once, the audio encoder is never run during training.

4. EXPERIMENTAL STUDY

4.1 Experimental setup

Test environment. All experiments run on a single workstation with an NVIDIA RTX 5090 GPU. The software stack is Python with PyTorch and the HuggingFace `transformers` library for the models, Optuna for hyperparameter search, and the `aac-metrics` package (with a Java runtime for its CoreNLP-backed metrics) for evaluation.

Data. We use MusicCaps [5] with a fixed three-way random split: a held-out test set ($n=529$, final metrics only), a validation set ($n\approx 495$, for checkpoint selection and hyperparameter search), and the remainder for training ($n\approx 4,455$). The split seed is fixed across all conditions, so every model and seed is scored on the identical test set.

Metrics. A single shared harness scores all conditions with the `aac-metrics` suite: BLEU-1 and BLEU-4 [17], METEOR [18], ROUGE-L [19], CIDEr-D [20], SPICE [21], SPIDER [22], SBERT-sim, FER and FENSE. FENSE [24] is our primary metric: it combines a Sentence-BERT [23] semantic similarity with a fluency-error penalty (FER), making it sensitive to the semantic correctness a listener cares about. We apply no post-processing by default; sentence trimming is treated as an explicit decoding ablation (Section 4.5).

Noise floor. To judge whether a delta is real, we retrain each baseline with three seeds (42, 43, 44), varying only initialisation, shuffling and dropout while holding the data split fixed. The standard deviation of each metric across seeds is our *noise floor*; deltas smaller than it are not interpreted as effects.

4.2 Baseline comparison

Figure 4.1 reports greedy-decoding test metrics with per-model noise floors, split into the **Legacy metrics** and **AAC metrics** groups following the `aac-metrics` taxonomy — the two groups live on different scales, so separate panels let each be inspected at its own resolution (exact values in Tables A.7 and A.8 in the appendix). GPT-2 leads on nine of the ten metrics, including the primary FENSE (0.3998 vs. 0.3954); the exception is FER, where T5 is lower (better), though that gap (0.023) sits inside T5’s own FER noise floor (± 0.0511). The FENSE margin (0.004) is likewise smaller than either model’s FENSE noise floor (± 0.0137 for GPT-2, ± 0.0345 for T5). We therefore do *not* read it as evidence that one architecture is better at music captioning under this recipe. The noise floor is also $\sim 2.5\times$ larger for T5 than GPT-2, so the encoder-decoder is noticeably noisier run-to-run and each model must be judged against its own variance.

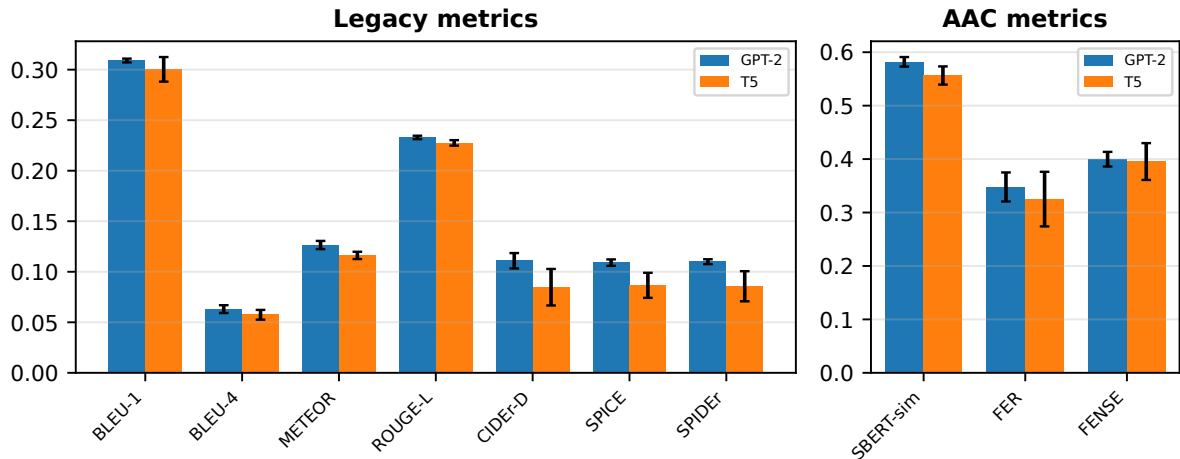


Figure 4.1: Baseline (greedy) test-set metrics per model, grouped into **Legacy metrics** (left) and **AAC metrics** (right), each panel at its own scale. Error bars show the per-model run-to-run standard deviation (noise floor) where measured. All metrics are higher-is-better except FER (lower is better).

4.3 Stage 1 vs. Stage 2

Figure 4.2 shows validation loss per epoch for every projection configuration, and Table 4.1 summarises the per-configuration best values. In Stage 1, where only the projection is trained, the configurations spread widely (a range of 0.38 for GPT-2 and 0.83 for T5). After Stage 2 fine-tuning that spread collapses to 0.03 for both models — an $11\times$ reduction for GPT-2 and $31\times$ for T5: full fine-tuning largely erases the architectural differences introduced by the projection. The S2 curves also reveal early overfitting — validation loss bottoms within a few epochs and then rises — which early stopping handles.

Table 4.1: Best validation loss per configuration at Stage 1 (projection only) and Stage 2 (full fine-tuning). The spread row shows the S1 spread collapsing after S2 fine-tuning (GPT-2 $11\times$, T5 $31\times$).

Config	GPT-2 S1	GPT-2 S2	T5 S1	T5 S2
baseline	2.6480	2.0573	2.4293	1.8903
depth1	2.8870	2.0530	3.0239	1.8929
depth3	2.6068	2.0609	2.3263	1.8674
drop0.1	2.6821	2.0570	2.3150	1.8941
drop0.5	2.6138	2.0554	2.2927	1.8855
prefix16	2.5091	2.0861	2.1973	1.8793
prefix4	2.8041	2.0601	2.4628	1.8900
Spread	0.3779	0.0332	0.8266	0.0267

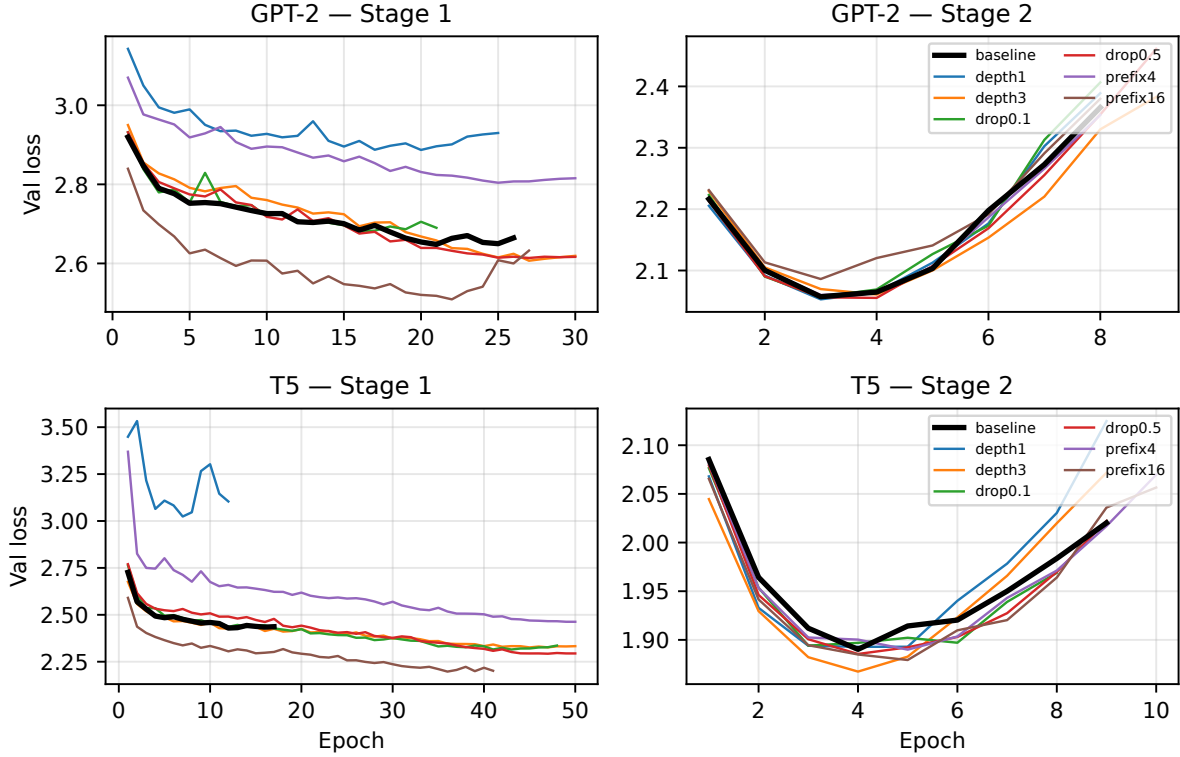


Figure 4.2: Validation loss per epoch for all projection configurations. *Left:* Stage 1 (projection only) shows a wide spread across configurations. *Right:* Stage 2 (full fine-tuning) collapses that spread – the configurations become nearly indistinguishable – and shows the early-overfitting onset. The baseline is drawn in bold.

4.4 Architecture ablations

We vary prefix length ($\{4, 16\}$ around 8), dropout ($\{0.1, 0.5\}$ around 0.3) and projection depth ($\{1, 3\}$ around 2), retraining each from scratch with greedy decoding. Figure 4.3 plots the resulting FENSE against the baseline $\pm$ noise-floor band. Consistent with Section 4.3, most configurations fall within the band: no projection hyperparameter reliably moves the primary metric for GPT-2. For T5 the prefix variants produce larger FENSE swings, but – as the next section shows – these coincide with collapses in the fluency-error term rather than genuine semantic gains. Full per-configuration metric tables are given in the appendix.

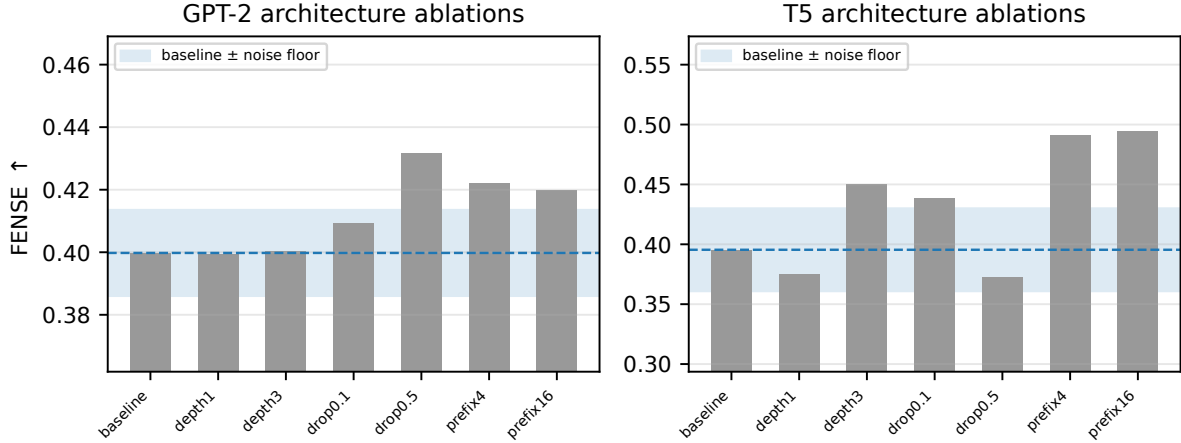


Figure 4.3: Architecture-ablation FENSE per configuration. The shaded band is the baseline $\pm$ one noise-floor standard deviation. Bars inside the band are not distinguishable from the baseline.

4.5 Decoding ablations

Decoding strategies are evaluated on the fixed baseline checkpoint (no retraining), isolating generation behaviour. Figure 4.5 shows the top configurations by FENSE per model, split into **Legacy metrics**, **AAC metrics** and a separate FER panel so each group can be read at its own resolution (full tables in the appendix, Tables A.15–A.18). The dominant lever is trimming trailing incomplete sentences: it lifts FENSE from 0.3998 to 0.5796 for GPT-2 and from 0.3954 to 0.5762 for T5.

This gain is, however, *largely an artifact of the metric*. Figure 4.4 plots FENSE against FER across all decoding configurations: the relationship is almost perfectly inverse, and the trimmed configurations simply drive FER to near zero while the lexical metrics in Tables A.15 and A.17 barely move. In other words, trimming removes the fluency penalty inside FENSE without making captions semantically more correct. We report it as a useful decoding choice for producing clean captions, but caution against reading the headline FENSE jump as a semantic improvement.

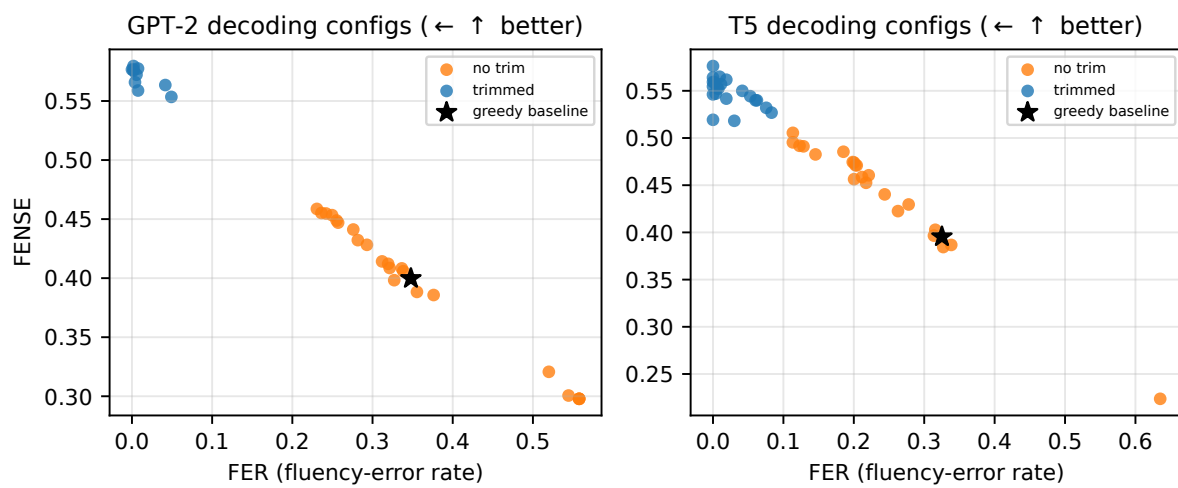


Figure 4.4: FENSE vs. FER over all decoding configurations. The near-perfect inverse trend shows that FENSE here is dominated by the fluency-error term: trimmed configurations (low FER) score high FENSE without improving semantic similarity. The star is greedy decoding.

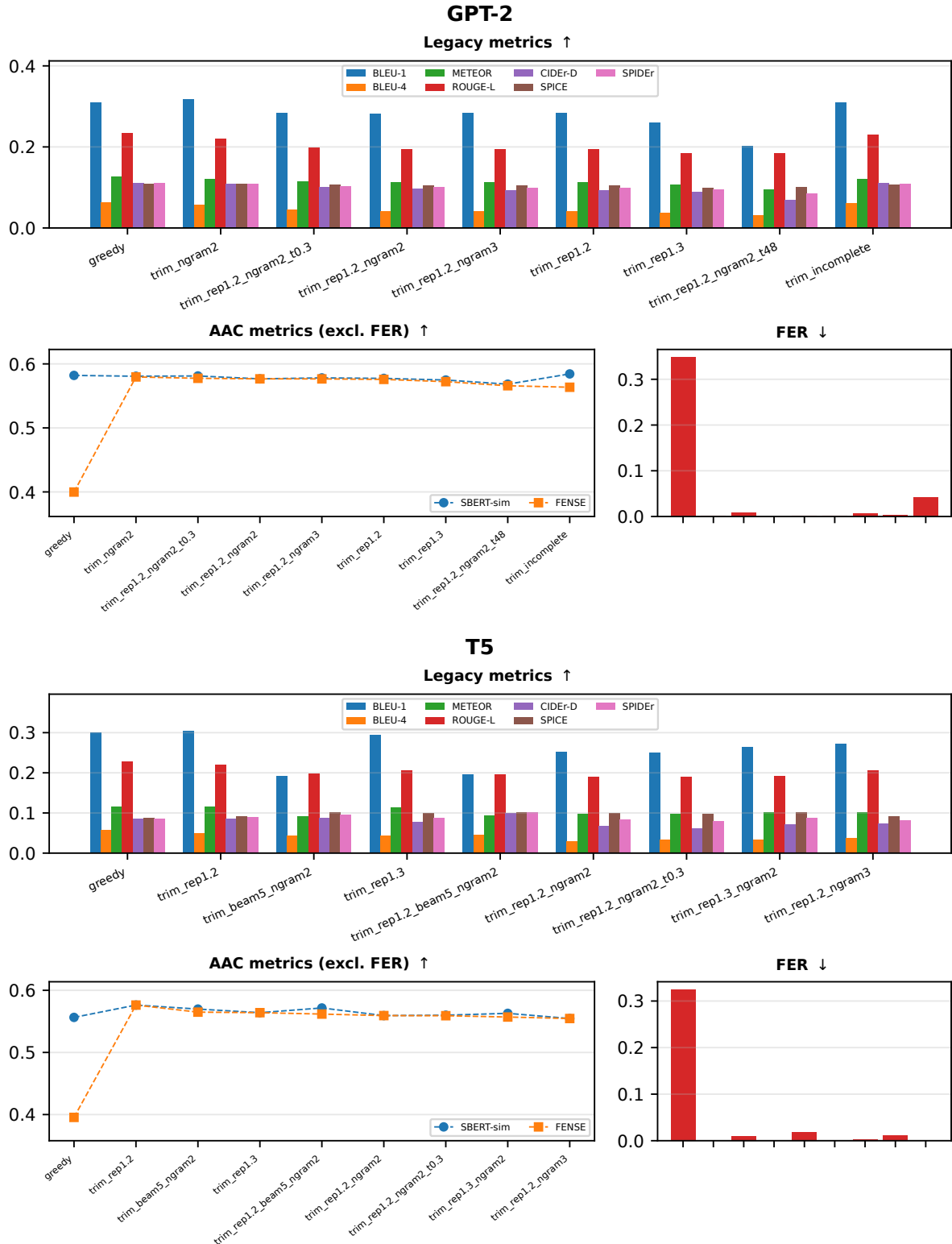


Figure 4.5: Decoding-ablation metrics per model (greedy baseline plus top configurations by FENSE). Each model section shows **Legacy metrics** (top, full width), with **AAC metrics excluding FER** (bottom left) and **FER** (bottom right) beneath, each panel at its own scale. The AAC panel uses markers on a zoomed (non-zero) axis to resolve the narrow 0.55–0.58 band of the trimmed configurations; FER is separated because its near-zero trimmed values would otherwise compress that axis, and its bars follow the same configuration order as the adjacent panel. The flat SBERT-sim trace next to the jumping FENSE trace and collapsing FER bars visualises the trimming artifact of Figure 4.4.

4.6 Qualitative analysis

Table 4.2 shows representative successes and failures. The models reliably recover high-level musical content – genre (rock, jazz, electronic), instrumentation (electric guitar, bass, drums, piano) and mood (groovy, energetic, emotional). Two failure modes recur: on *non-music* audio the pipeline still emits a confident music caption, and greedy decoding occasionally degenerates into verbatim repetition. Notably, degeneration and semantic accuracy co-occur: in the bowl-tone example both models correctly capture the sound itself – a resonating bowl, rubbed or struck, with a meditative mood – yet both repeat themselves, T5 collapsing into the same sentence four times over and GPT-2 duplicating its final sentence. On non-music audio the spurious male vocalist appears only in GPT-2’s printer caption; T5 instead invents a plausible non-vocal scene (a DJ scratching). Most strikingly, both models tend to insert “a male vocalist singing melodically” even when the reference explicitly states there is no singer.

To characterise this we measured vocal-presence and vocalist-gender agreement over the full test set (full statistics in Tables A.9 and A.10 in the appendix). The bias is *not* a blind spot: GPT-2 mentions vocals on 95% of clips whose reference describes singing, but also on 43% of clips marked instrumental (T5: 82% / 29%) – both models track vocal presence well above chance while skewing toward false positives. The skew mirrors the training captions, of which 53% mention vocals and in which male vocalists outnumber female ones 2.2:1; the phrase “male vocalist” appears in 10% of training captions but in roughly half of the generated ones. Gender is worse: on references describing a *female* singer, T5 outputs “male” 89% of the time and GPT-2 44%, while male references are almost never misgendered (2–4%). These rates persist across decoding configurations, so they are not a decoding artifact.

To separate the frontend from the LM, we probed the frozen CLAP embeddings directly with zero-shot text prompts: vocal vs. instrumental is separable at 74% accuracy and vocalist gender at 76% (female recall only 55%). The embedding therefore carries the relevant cues, but noisily – and since both LMs read the *same* embedding, the gap between GPT-2 (56%) and T5 (11%) in female-vocalist accuracy shows the failure is largely model-side. We conclude that the vocalist bias is caption-prior amplification under uncertain audio evidence, not a missing audio cue: a music-specialised encoder should sharpen the evidence, but the male-vocalist default itself is learned from the caption statistics.

Table 4.2: Representative successes and failures (baseline greedy; captions trimmed at the last full sentence for display). The models recover genre, instrumentation and mood, but default to a male vocalist on instrumental audio (and GPT-2 also on the non-music printer clip). In the second failure case both models capture the sound semantics correctly yet degenerate into repetition.

Successes	
<i>Reference</i>	instrumental rock music with a drum kit, lots of cymbal crashes, acoustic guitar strumming, harmonized lead guitars and an active bass guitar.
<i>GPT-2</i>	This is a rock music piece. There is a male vocalist singing melodically. The electric guitar is playing the main tune while the bass guitar is playing in the background. The rhythmic background consists of a simple acoustic drum beat. The atmosphere is groovy.
<i>T5</i>	This is a rock music piece. There is a male vocalist singing melodically. The electric guitar is playing a simple tune with a bass guitar playing in the background. The rhythmic background consists of a fast-paced acoustic drum beat.
<i>Reference</i>	This is a drum & bass electronic music piece. There is no singer. There is a strong and fuzzy feeling bass sound. The rhythmic background consists of a loud and high tempo electronic drum beat. The atmosphere is modern and energetic. This piece could be playing at a nightclub or a dance club.
<i>GPT-2</i>	This is a techno music piece. There is a male vocalist singing melodically. The melody is being played by the synth bass. The rhythmic background consists of a fast-paced electronic drum beat. The atmosphere is groovy. This piece could be used in the background of nightclubs.
<i>T5</i>	This is a techno house music piece. There is a male vocalist singing melodically. The melody is being played by the synth bass while the rhythmic background consists of a fast-paced electronic drum beat. The atmosphere is energetic.
<i>Reference</i>	A female vocalist sings this melancholic melody in a foreign language. The tempo is slow with a romantic piano accompaniment. The song is soft, mellow, poignant, emotional, sentimental, romantic, melancholic, sad, lonely, and wistful. This song is a Pop/ Love song.
<i>GPT-2</i>	This is a lullaby piece. There is a female vocalist singing melodically. The piano is playing a simple tune while the bass guitar is playing in the background. The atmosphere is calming and relaxing. This piece could be used in the background of a meditation video.
<i>T5</i>	This is a classical music piece. There is a female vocalist singing melodically. The melody is being played by the piano while the rhythm is being played by the acoustic guitar. The atmosphere is romantic.
<i>Reference</i>	The low quality recording features a live performance of a jazz song and it consists of a groovy bass guitar, groovy rhodes keys, electric guitar melody, shimmering hi hats, punchy kick and punchy snare hits. It sounds groovy, funky and passionate. The recording is noisy and in mono.
<i>GPT-2</i>	This is a jazz music piece. There is a saxophone solo being played. The melody is being played by the piano and the electric guitar. The rhythmic background consists of a simple acoustic drum beat. The atmosphere is groovy. This piece could be used in the background of a jazz bar.
<i>T5</i>	This is a live recording of a jazz music piece. There is a saxophone playing the lead melody while a bass guitar is playing in the background. The rhythm is being played by the acoustic drums. The atmosphere is jazzy.
Failure cases	
<i>Reference</i>	The low quality recording features a printer that produces printing sounds and some muffled music playing in the background, probably on a radio. It is noisy and in mono.
<i>GPT-2</i>	This is a recording of a live performance of a pop music piece. There is a male vocalist singing melodically. The melody is being played by a keyboard and a bass guitar while the rhythmic background is provided by a fast tempo electronic drum beat.
<i>T5</i>	This is a live recording of a DJ performance. The DJ is scratching the turntable repeatedly. The turntable is a low quality recording. The sound of the turntable is distorted and the recording is of very poor quality.
<i>Reference</i>	The low quality recording features a metallic hit, followed by resonating bowl tone. The recording is mono and noisy.
<i>GPT-2</i>	This is a meditation music piece. There is a bowl-like sound being played by the bowl. The bowl is being rubbed repeatedly. There is a calming and relaxing atmosphere to this piece. This piece can be used in the background of meditation videos. It can also be used in the background of meditation videos.
<i>T5</i>	This is a Tibetan singing bowl playing in the background of a meditation video. There is a chanting bowl being played. There is a chanting bowl being played. There is a chanting bowl being played. There is a chanting bowl being played.

5. CONCLUSIONS AND FUTURE WORK

We presented CLAPCAP, a controlled study of lightweight language models for *music* captioning via an audio adaptation of the ClipCap prefix recipe. Holding the frozen CLAP encoder, the projection, the data split and the evaluation fixed, we found that a decoder-only and an encoder–decoder LM are statistically indistinguishable at baseline once run-to-run variance is measured; that second-stage fine-tuning erases projection-level differences; and that the largest decoding lever improves the primary metric mostly by removing a fluency penalty rather than by improving semantics. A reproducible vocalist bias turns out to be caption-prior amplification under uncertain audio evidence. The results characterise the minimum viable architecture for music captioning and the care needed to interpret its metrics, rather than chasing state of the art.

5.1 Limitations

General-purpose audio frontend. Our encoder is trained on general audio, not music. A music-specialised encoder (a music CLAP checkpoint, MERT [25] or MusicFM [26]) is a natural and likely beneficial swap, especially for the fine-grained attributes (vocalist gender, instrument identity) where the embedding’s cues are noisiest (Section 4.6).

Capacity vs. data. Scaling the LM is not free in this low-data regime (~ 4.5 k clips). In preliminary runs a billion-parameter decoder fine-tuned end-to-end overfits within a single epoch — the same MLP-plus-fine-tuning overfitting that ClipCap [1] reports, amplified by scale — and requires a gentler schedule to remain comparable. This supports the lightweight framing and motivates our in-progress extension to such models.

Metric caveats. MusicCaps provides a single reference per clip, which mechanically depresses n-gram metrics relative to multi-reference benchmarks. And as Section 4.5 shows, FENSE can be inflated by fluency-driven trimming; we mitigate this by reporting FER and lexical metrics alongside it.

5.2 Future work

Future directions include: (i) extending the controlled comparison to modern billion-parameter decoders with the gentler fine-tuning schedule identified in preliminary runs; (ii) swapping the general-purpose CLAP frontend for a music-specialised encoder; (iii) a leakage-free comparison against a domain-specific music captioner evaluated on our exact test split via the same harness; (iv) an LLM-as-judge and human evaluation of semantic correctness, reporting inter-rater agreement; and (v) joint hyperparameter optimisation over projection and decoding parameters, replacing the single-variable reconnaissance ablations.

ALIGNMENT WITH UN SUSTAINABLE DEVELOPMENT GOALS

Impact Area	Does project impact? Direct or indirect?	Impact Definition
Social (How does the project affect people and society? Who benefits from the project? Does it improve accessibility, inclusion, or equality? Does it reduce manual effort or improve quality of life?) SDG 10 - Reduced Inequalities	Yes, indirect	Textual music descriptions improve accessibility for deaf and hard-of-hearing users and enable text-based music search. The project also quantifies a gender bias in generated captions (male-vocalist default, female misgendering) and traces its cause, supporting fairer future captioning systems.
Health (How does the project affect health directly or indirectly? Does it reduce stress, workload, or human error? Does it support health monitoring, safety, or well-being? Does it prevent unsafe working conditions?) SDG 3 - Good Health and Well-being	Limited, indirect	Automatic captioning reduces the manual annotation workload for music catalogues; no direct health impact.
Security (How is security achieved in the project, , or how does the project increase the security? How is user data protected? What security risks exist?) SDG 16 - Peace, Justice, and Strong Institutions	Yes, indirect	No personal data is processed: the project uses the public MusicCaps research dataset only. Code and models are open, so results are auditable. The main risk is biased output, which the report measures and documents explicitly rather than hiding.
Economic (How does the project contribute to the economy? Does it reduce costs? Does it improve productivity?) SDG 8 - Decent Work and Economic Growth	Yes, indirect	The lightweight recipe (124M–220M parameters, ~10M newly trained) runs on a single consumer GPU, lowering the cost barrier for music-tech applications and research compared to billion-parameter end-to-end audio LLMs. Automatic captions reduce manual labelling cost.
Sustainability and Environment (How does the project contribute to the sustainability, and affect the environment? Does it reduce paper usage? Does it optimize energy or resource consumption? Is computer system usage efficient?) SDG 11 - Sustainable cities and communities SDG 13 - Climate Action SDG 12 - Responsible Consumption and Production	Yes, direct	Efficiency is a core design goal: small models, frozen encoder with precomputed embeddings (the audio encoder never runs during training), and eval-only decoding ablations minimise GPU hours and energy versus training large end-to-end models.

REFERENCES

- [1] R. Mokady, A. Hertz, and A. H. Bermano, "ClipCap: CLIP Prefix for Image Captioning," *arXiv:2111.09734*, 2021. 3, 4, 15
- [2] C. D. Kim, B. Kim, H. Lee, and G. Kim, "AudioCaps: Generating Captions for Audios in The Wild," in *Proc. NAACL-HLT*, 2019. 3, 4
- [3] K. Drossos, S. Lipping, and T. Virtanen, "Clotho: An Audio Captioning Dataset," in *Proc. ICASSP*, 2020. 3, 4
- [4] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-Scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation," in *Proc. ICASSP*, 2023. 3, 5
- [5] A. Agostinelli, T. I. Denk, Z. Borsos, et al., "MusicLM: Generating Music From Text," *arXiv:2301.11325*, 2023. 4, 7
- [6] S. Deshmukh, B. Elizalde, R. Singh, and H. Wang, "Pengi: An Audio Language Model for Audio Tasks," in *Proc. NeurIPS*, 2023. 4
- [7] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An Ontology and Human-Labeled Dataset for Audio Events," in *Proc. ICASSP*, 2017. 4
- [8] S. Doh, K. Choi, J. Lee, and J. Nam, "LP-MusicCaps: LLM-Based Pseudo Music Captioning," in *Proc. ISMIR*, 2023. 4
- [9] Y. Chu, J. Xu, X. Zhou, et al., "Qwen-Audio: Advancing Universal Audio Understanding via Unified Large-Scale Audio-Language Models," *arXiv:2311.07919*, 2023. 4
- [10] C. Tang, W. Yu, G. Sun, et al., "SALMONN: Towards Generic Hearing Abilities for Large Language Models," in *Proc. ICLR*, 2024. 4
- [11] K. Chen, X. Du, B. Zhu, Z. Ma, T. Berg-Kirkpatrick, and S. Dubnov, "HTS-AT: A Hierarchical Token-Semantic Audio Transformer for Sound Classification and Detection," in *Proc. ICASSP*, 2022. 5
- [12] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI, Tech. Rep., 2019. 5
- [13] C. Raffel, N. Shazeer, A. Roberts, et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *JMLR*, vol. 21, no. 140, 2020. 5
- [14] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019. 6
- [15] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *Proc. ICLR*, 2017. 6
- [16] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-Generation Hyperparameter Optimization Framework," in *Proc. ACM SIGKDD*, 2019. 6
- [17] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," in *Proc. ACL*, 2002. 7

-
- [18] S. Banerjee and A. Lavie, "METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments," in *Proc. ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 2005. 7
 - [19] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out (ACL Workshop)*, 2004. 7
 - [20] R. Vedantam, C. L. Zitnick, and D. Parikh, "CIDEr: Consensus-Based Image Description Evaluation," in *Proc. CVPR*, 2015. 7
 - [21] P. Anderson, B. Fernando, M. Johnson, and S. Gould, "SPICE: Semantic Propositional Image Caption Evaluation," in *Proc. ECCV*, 2016. 7
 - [22] S. Liu, Z. Zhu, N. Ye, S. Guadarrama, and K. Murphy, "Improved Image Captioning via Policy Gradient Optimization of SPIDer," in *Proc. ICCV*, 2017. 7
 - [23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, 2019. 7
 - [24] Z. Zhou, Z. Zhang, X. Xu, et al., "Can Audio Captions Be Evaluated with Image Caption Metrics?," in *Proc. ICASSP*, 2022. 7
 - [25] Y. Li, R. Yuan, G. Zhang, et al., "MERT: Acoustic Music Understanding Model with Large-Scale Self-Supervised Training," in *Proc. ICLR*, 2024. 15
 - [26] M. Won, Y.-N. Hung, and D. Le, "A Foundation Model for Music Informatics," in *Proc. ICASSP*, 2024. 15

A. APPENDIX

A.1 Reproducibility

All code, configuration files and table/figure generation scripts are available at github.com/raccoon-42/audio-caption. The pipeline is organised as one script per step: data preparation (CLAP embedding precomputation), per-model learning-rate search, baseline training, noise-floor seed replication, architecture ablations, decoding ablations and shared evaluation. Dependencies are managed with `uv`; evaluation additionally requires a Java runtime (JDK 11) and a one-time `aac-metrics-download` for the CoreNLP-backed metrics. Every table and figure in this report is generated from the result JSONs by the scripts under `scripts/reporting/`: `thesis_tables.py` and `figures.py`; the vocalist-bias statistics come from `scripts/reporting/vocal_bias.py`.

A.2 Noise-floor detail

Per-seed metric values behind the noise floors of Tables A.7 and A.8.

Table A.1: GPT-2 run-to-run variance over seeds [42, 43, 44] (std = noise floor).

Seed	BLEU-1 ↑	BLEU-4 ↑	METEOR ↑	ROUGE-L ↑	CIDEr-D ↑	SPICE ↑	SPIDEr ↑	SBERT-sim ↑	FER ↓	FENSE ↑
42	0.3091	0.0631	0.1265	0.2329	0.1109	0.1091	0.1100	0.5820	0.3478	0.3998
43	0.3126	0.0702	0.1338	0.2361	0.1254	0.1027	0.1141	0.5982	0.3573	0.4095
44	0.3097	0.0692	0.1332	0.2356	0.1220	0.1064	0.1142	0.5966	0.3989	0.3824
std	0.0019	0.0039	0.0041	0.0017	0.0076	0.0032	0.0024	0.0089	0.0272	0.0137

Table A.2: T5 run-to-run variance over seeds [42, 43, 44] (std = noise floor).

Seed	BLEU-1 ↑	BLEU-4 ↑	METEOR ↑	ROUGE-L ↑	CIDEr-D ↑	SPICE ↑	SPIDEr ↑	SBERT-sim ↑	FER ↓	FENSE ↑
42	0.3003	0.0574	0.1162	0.2275	0.0847	0.0866	0.0857	0.5564	0.3251	0.3954
43	0.3164	0.0663	0.1234	0.2314	0.0961	0.1032	0.0996	0.5736	0.3724	0.3790
44	0.2926	0.0651	0.1201	0.2328	0.1201	0.1108	0.1155	0.5905	0.2703	0.4452
std	0.0122	0.0048	0.0036	0.0027	0.0181	0.0124	0.0149	0.0170	0.0511	0.0345

A.3 Stage-wise validation losses

Best validation loss per configuration, shown separately for Stage 1 and Stage 2 (the spread collapse discussed in Section 4.3).

GPT-2

Table A.3: GPT-2 stage-1 best validation loss per config (spread = 0.3779).

Config	Best val loss	Stopped epoch
gpt2	2.6480	26
gpt2_depth1	2.8870	25
gpt2_depth3	2.6068	30
gpt2_drop0.1	2.6821	21
gpt2_drop0.5	2.6138	30
gpt2_prefix16	2.5091	27
gpt2_prefix4	2.8041	30

Table A.4: GPT-2 stage-2 best validation loss per config (spread = 0.0332).

Config	Best val loss	Stopped epoch
gpt2	2.0573	8
gpt2_depth1	2.0530	8
gpt2_depth3	2.0609	9
gpt2_drop0.1	2.0570	8
gpt2_drop0.5	2.0554	9
gpt2_prefix16	2.0861	8
gpt2_prefix4	2.0601	8

T5

Table A.5: T5 stage-1 best validation loss per config (spread = 0.8266).

Config	Best val loss	Stopped epoch
t5	2.4293	17
t5_depth1	3.0239	12
t5_depth3	2.3263	50
t5_drop0.1	2.3150	48
t5_drop0.5	2.2927	50
t5_prefix16	2.1973	41
t5_prefix4	2.4628	50

Table A.6: T5 stage-2 best validation loss per config (spread = 0.0267).

Config	Best val loss	Stopped epoch
t5	1.8903	9
t5_depth1	1.8929	9
t5_depth3	1.8674	9
t5_drop0.1	1.8941	8
t5_drop0.5	1.8855	9
t5_prefix16	1.8793	10
t5_prefix4	1.8900	10

A.4 Baseline metric tables

Exact values behind Figure 4.1, split into **Legacy metrics** and **AAC metrics**.

Table A.7: Baseline (greedy) test-set metrics — Legacy metrics. Values show $\pm$ run-to-run standard deviation (noise floor) where measured.

Model	BLEU-1 $\uparrow$	BLEU-4 $\uparrow$	METEOR $\uparrow$	ROUGE-L $\uparrow$	CIDEr-D $\uparrow$	SPICE $\uparrow$	SPIDEr $\uparrow$
GPT-2	0.3091 $\pm$ 0.0019	0.0631 $\pm$ 0.0039	0.1265 $\pm$ 0.0041	0.2329 $\pm$ 0.0017	0.1109 $\pm$ 0.0076	0.1091 $\pm$ 0.0032	0.1100 $\pm$ 0.0024
T5	0.3003 $\pm$ 0.0122	0.0574 $\pm$ 0.0048	0.1162 $\pm$ 0.0036	0.2275 $\pm$ 0.0027	0.0847 $\pm$ 0.0181	0.0866 $\pm$ 0.0124	0.0857 $\pm$ 0.0149

$\uparrow$: higher is better; $\downarrow$: lower is better.

Table A.8: Baseline (greedy) test-set metrics — AAC-specific metrics. Values show $\pm$ run-to-run standard deviation (noise floor) where measured.

Model	SBERT-sim $\uparrow$	FER $\downarrow$	FENSE $\uparrow$
GPT-2	0.5820 $\pm$ 0.0089	0.3478 $\pm$ 0.0272	0.3998 $\pm$ 0.0137
T5	0.5564 $\pm$ 0.0170	0.3251 $\pm$ 0.0511	0.3954 $\pm$ 0.0345

$\uparrow$: higher is better; $\downarrow$: lower is better.

A.5 Vocalist-bias statistics

Full statistics behind the bias analysis of Section 4.6: per-model vocal-presence and gender confusion rates, the training-caption prior, and the zero-shot probe of the frozen audio embeddings.

Table A.9: Vocal-presence and vocalist-gender statistics per model (baseline greedy predictions, $n = 529$ test clips). Gender rows count only captions that mention exactly one gender.

	GPT-2	T5
Vocal mention on vocal reference	234/247 (95%)	203/247 (82%)
Vocal mention on instrumental reference	49/113 (43%)	33/113 (29%)
Overall vocal-mention rate	65%	53%
“male vocalist” phrase rate	52%	47%
Female ref. → female caption	40/71 (56%)	7/64 (11%)
Female ref. → male caption	31/71 (44%)	57/64 (89%)
Male ref. → male caption	130/136 (96%)	114/116 (98%)
Male ref. → female caption	6/136 (4%)	2/116 (2%)

Table A.10: Model-independent context for the vocalist bias: training-caption prior over all MusicCaps captions, and zero-shot probes of the frozen CLAP audio embeddings against caption-derived labels.

Corpus prior ($n = 5292$ captions)	Vocal mention	53%
	Male-only gender mention	28%
	Female-only gender mention	13%
	Male:female ratio	2.2:1
	“male vocalist” phrase	10%
CLAP probe: vocal/instr. ($n = 3556$)	Accuracy	74%
	Recall (vocal)	77%
	Recall (instrumental)	65%
CLAP probe: gender ($n = 2164$)	Accuracy	76%
	Recall (male)	85%
	Recall (female)	55%

A.6 Full architecture-ablation metrics

GPT-2

Table A.11: GPT-2 architecture ablations (greedy) – Legacy metrics. Best per column in bold; baseline row shows $\pm$ noise floor.

Config	BLEU-1 $\uparrow$	BLEU-4 $\uparrow$	METEOR $\uparrow$	ROUGE-L $\uparrow$	CIDEr-D $\uparrow$	SPICE $\uparrow$	SPIDEr $\uparrow$
gpt2	0.3091 $\pm$ 0.0019	0.0631 $\pm$ 0.0039	0.1265 $\pm$ 0.0041	0.2329 $\pm$ 0.0017	0.1109 $\pm$ 0.0076	0.1091 $\pm$ 0.0032	0.1100 $\pm$ 0.0024
gpt2_depth1	0.3103	0.0631	0.1288	0.2339	0.1078	0.1164	0.1121
gpt2_depth3	0.2951	0.0600	0.1255	0.2201	0.0952	0.1000	0.0976
gpt2_drop0.1	0.3106	0.0696	0.1337	0.2371	0.1268	0.1152	0.1210
gpt2_drop0.5	0.3052	0.0653	0.1297	0.2317	0.1080	0.1050	0.1065
gpt2_prefix16	0.3093	0.0698	0.1324	0.2345	0.1235	0.1118	0.1177
gpt2_prefix4	0.3101	0.0638	0.1307	0.2327	0.1020	0.1004	0.1012

$\uparrow$: higher is better; $\downarrow$: lower is better.

Table A.12: GPT-2 architecture ablations (greedy) – AAC-specific metrics. Best per column in bold; baseline row shows $\pm$ noise floor.

Config	SBERT-sim $\uparrow$	FER $\downarrow$	FENSE $\uparrow$
gpt2	0.5820 $\pm$ 0.0089	0.3478 $\pm$ 0.0272	0.3998 $\pm$ 0.0137
gpt2_depth1	0.5846	0.3535	0.3992
gpt2_depth3	0.5858	0.3535	0.4002
gpt2_drop0.1	0.6029	0.3516	0.4092
gpt2_drop0.5	0.5896	0.2987	0.4316
gpt2_prefix16	0.5943	0.3308	0.4199
gpt2_prefix4	0.5858	0.3100	0.4221

$\uparrow$: higher is better; $\downarrow$: lower is better.

T5

Table A.13: T5 architecture ablations (greedy) – Legacy metrics. Best per column in bold; baseline row shows $\pm$ noise floor.

Config	BLEU-1 $\uparrow$	BLEU-4 $\uparrow$	METEOR $\uparrow$	ROUGE-L $\uparrow$	CIDEr-D $\uparrow$	SPICE $\uparrow$	SPIDEr $\uparrow$
t5	0.3003 $\pm$ 0.0122	0.0574 $\pm$ 0.0048	0.1162 $\pm$ 0.0036	0.2275 $\pm$ 0.0027	0.0847 $\pm$ 0.0181	0.0866 $\pm$ 0.0124	0.0857 $\pm$ 0.0149
t5_depth1	0.2947	0.0546	0.1135	0.2267	0.0893	0.0953	0.0923
t5_depth3	0.2400	0.0549	0.1044	0.2111	0.0968	0.0963	0.0965
t5_drop0.1	0.2828	0.0570	0.1154	0.2291	0.1308	0.1044	0.1176
t5_drop0.5	0.2877	0.0543	0.1143	0.2255	0.0971	0.0999	0.0985
t5_prefix16	0.2508	0.0610	0.1095	0.2191	0.1263	0.1105	0.1184
t5_prefix4	0.2683	0.0655	0.1157	0.2271	0.1240	0.1139	0.1190

$\uparrow$: higher is better; $\downarrow$: lower is better.

Table A.14: T5 architecture ablations (greedy) – AAC-specific metrics. Best per column in bold; baseline row shows $\pm$ noise floor.

Config	SBERT-sim $\uparrow$	FER $\downarrow$	FENSE $\uparrow$
t5	0.5564 $\pm$ 0.0170	0.3251 $\pm$ 0.0511	0.3954 $\pm$ 0.0345
t5_depth1	0.5702	0.3781	0.3755
t5_depth3	0.5759	0.2476	0.4500
t5_drop0.1	0.5883	0.2836	0.4389
t5_drop0.5	0.5764	0.3913	0.3725
t5_prefix16	0.5947	0.1853	0.4947
t5_prefix4	0.5898	0.1890	0.4908

$\uparrow$: higher is better; $\downarrow$: lower is better.

A.7 Full decoding-ablation metrics

Exact values behind Figure 4.5, split into **Legacy metrics** and **AAC metrics**.

GPT-2

Table A.15: GPT-2 decoding ablations — top 15 by FENSE, Legacy metrics. The greedy row is the baseline; its $\pm$ is the run-to-run training noise floor, shown for context only — all rows decode the same checkpoint, so differences between configurations are exact.

Config	BLEU-1 $\uparrow$	BLEU-4 $\uparrow$	METEOR $\uparrow$	ROUGE-L $\uparrow$	CIDEr-D $\uparrow$	SPICE $\uparrow$	SPIDEr $\uparrow$
greedy	0.3091 $\pm$ 0.0019	0.0631 $\pm$ 0.0039	0.1265 $\pm$ 0.0041	0.2329 $\pm$ 0.0017	0.1109 $\pm$ 0.0076	0.1091 $\pm$ 0.0032	0.1100 $\pm$ 0.0024
trim_ngram2	0.3171	0.0575	0.1210	0.2205	0.1092	0.1081	0.1087
trim_rep1.2_ngram2_t0.3	0.2829	0.0454	0.1137	0.1970	0.1003	0.1062	0.1033
trim_rep1.2_ngram2	0.2810	0.0409	0.1124	0.1931	0.0959	0.1042	0.1000
trim_rep1.2_ngram3	0.2839	0.0407	0.1127	0.1944	0.0926	0.1045	0.0986
trim_rep1.2	0.2834	0.0408	0.1124	0.1945	0.0924	0.1043	0.0983
trim_rep1.3	0.2605	0.0374	0.1058	0.1843	0.0885	0.0990	0.0938
trim_rep1.2_ngram2_t48	0.2011	0.0308	0.0938	0.1835	0.0692	0.1002	0.0847
trim_incomplete	0.3091	0.0605	0.1202	0.2303	0.1106	0.1071	0.1088
trim_rep1.8_ngram2_t0.3	0.2252	0.0348	0.0964	0.1708	0.0725	0.0940	0.0832
trim_t48	0.2352	0.0433	0.1028	0.2140	0.0911	0.1010	0.0960
rep1.2_ngram3	0.2869	0.0415	0.1212	0.1944	0.0900	0.1046	0.0973
rep_1.2	0.2863	0.0417	0.1214	0.1952	0.0890	0.1056	0.0973
rep_1.3	0.2794	0.0385	0.1188	0.1867	0.0842	0.1025	0.0934
rep1.8_ngram2_t0.3	0.2723	0.0381	0.1185	0.1793	0.0920	0.1041	0.0980
rep_1.8	0.2691	0.0322	0.1162	0.1763	0.0754	0.1001	0.0877

$\uparrow$: higher is better; $\downarrow$: lower is better.

Table A.16: GPT-2 decoding ablations — top 15 by FENSE, AAC-specific metrics. The greedy row is the baseline; its $\pm$ is the run-to-run training noise floor, shown for context only — all rows decode the same checkpoint, so differences between configurations are exact.

Config	SBERT-sim $\uparrow$	FER $\downarrow$	FENSE $\uparrow$
greedy	0.5820 $\pm$ 0.0089	0.3478 $\pm$ 0.0272	0.3998 $\pm$ 0.0137
trim_ngram2	0.5807	0.0019	0.5796
trim_rep1.2_ngram2_t0.3	0.5812	0.0076	0.5775
trim_rep1.2_ngram2	0.5766	0.0000	0.5766
trim_rep1.2_ngram3	0.5781	0.0019	0.5766
trim_rep1.2	0.5774	0.0019	0.5758
trim_rep1.3	0.5748	0.0057	0.5723
trim_rep1.2_ngram2_t48	0.5683	0.0038	0.5659
trim_incomplete	0.5844	0.0416	0.5635
trim_rep1.8_ngram2_t0.3	0.5622	0.0076	0.5589
trim_t48	0.5775	0.0491	0.5535
rep1.2_ngram3	0.5812	0.2306	0.4586
rep_1.2	0.5811	0.2363	0.4552
rep_1.3	0.5818	0.2420	0.4548
rep1.8_ngram2_t0.3	0.5857	0.2495	0.4534
rep_1.8	0.5800	0.2552	0.4488

$\uparrow$: higher is better; $\downarrow$: lower is better.

T5

Table A.17: T5 decoding ablations – top 15 by FENSE, Legacy metrics. The greedy row is the baseline; its $\pm$ is the run-to-run training noise floor, shown for context only – all rows decode the same checkpoint, so differences between configurations are exact.

Config	BLEU-1 $\uparrow$	BLEU-4 $\uparrow$	METEOR $\uparrow$	ROUGE-L $\uparrow$	CIDEr-D $\uparrow$	SPICE $\uparrow$	SPIDEr $\uparrow$
greedy	0.3003 $\pm$ 0.0122	0.0574 $\pm$0.0048	0.1162 $\pm$0.0036	0.2275 $\pm$0.0027	0.0847 $\pm$ 0.0181	0.0866 $\pm$ 0.0124	0.0857 $\pm$ 0.0149
trim_rep1.2	0.3045	0.0498	0.1158	0.2203	0.0863	0.0921	0.0892
trim_beam5_ngram2	0.1923	0.0428	0.0918	0.1967	0.0878	0.1013	0.0945
trim_rep1.3	0.2934	0.0424	0.1124	0.2065	0.0779	0.0987	0.0883
trim_rep1.2_beam5_ngram2	0.1956	0.0442	0.0941	0.1949	0.0985	0.1023	0.1004
trim_rep1.2_ngram2	0.2527	0.0296	0.0982	0.1904	0.0662	0.1000	0.0831
trim_rep1.2_ngram2_t0.3	0.2500	0.0331	0.0978	0.1897	0.0611	0.0973	0.0792
trim_rep1.3_ngram2	0.2629	0.0331	0.1018	0.1926	0.0711	0.1019	0.0865
trim_rep1.2_ngram3	0.2725	0.0371	0.1017	0.2063	0.0738	0.0903	0.0821
trim_rep1.5	0.2579	0.0264	0.1010	0.1817	0.0653	0.0916	0.0785
trim_rep1.5_ngram2	0.2560	0.0261	0.1008	0.1803	0.0647	0.0918	0.0782
trim_rep1.3_beam5	0.2262	0.0563	0.1041	0.2116	0.1512	0.1099	0.1306
trim_ngram2	0.2406	0.0357	0.0969	0.1966	0.0569	0.0902	0.0736
trim_rep1.2_ngram2_t48	0.1981	0.0226	0.0867	0.1831	0.0474	0.0942	0.0708
trim_rep1.2_beam5_lp0.6	0.2236	0.0553	0.1034	0.2122	0.1396	0.1087	0.1241
trim_incomplete	0.2670	0.0476	0.1091	0.2180	0.0662	0.0854	0.0758

$\uparrow$: higher is better; $\downarrow$: lower is better.

Table A.18: T5 decoding ablations – top 15 by FENSE, AAC-specific metrics. The greedy row is the baseline; its $\pm$ is the run-to-run training noise floor, shown for context only – all rows decode the same checkpoint, so differences between configurations are exact.

Config	SBERT-sim $\uparrow$	FER $\downarrow$	FENSE $\uparrow$
greedy	0.5564 $\pm$ 0.0170	0.3251 $\pm$ 0.0511	0.3954 $\pm$ 0.0345
trim_rep1.2	0.5762	0.0000	0.5762
trim_beam5_ngram2	0.5698	0.0095	0.5649
trim_rep1.3	0.5640	0.0000	0.5640
trim_rep1.2_beam5_ngram2	0.5716	0.0189	0.5618
trim_rep1.2_ngram2	0.5593	0.0000	0.5593
trim_rep1.2_ngram2_t0.3	0.5599	0.0019	0.5591
trim_rep1.3_ngram2	0.5629	0.0113	0.5570
trim_rep1.2_ngram3	0.5546	0.0000	0.5546
trim_rep1.5	0.5561	0.0057	0.5533
trim_rep1.5_ngram2	0.5558	0.0076	0.5522
trim_rep1.3_beam5	0.5728	0.0416	0.5500
trim_ngram2	0.5485	0.0038	0.5470
trim_rep1.2_ngram2_t48	0.5462	0.0000	0.5462
trim_rep1.2_beam5_lp0.6	0.5716	0.0529	0.5443
trim_incomplete	0.5495	0.0189	0.5417

$\uparrow$: higher is better; $\downarrow$: lower is better.